

FireGuard IoT — Slides de Apresentação

Slide 1: Capa

Título: FireGuard IoT: Sistema Inteligente de Alarme de Incêndio

Subtítulo: Trabalho 2 — Integração de Hardware, Backend e Frontend

Autores: Richardson, Wallace, Emanuele, Vinícius

Data: Junho de 2026

Slide 2: Visão Geral do Projeto

Título: O que é FireGuard IoT?

FireGuard IoT é um sistema completo de monitoramento e alarme de incêndio que integra três camadas tecnológicas: (1) **Hardware Arduino** que monitora continuamente o ambiente através de um sensor de chama infravermelho, (2) **Backend Python** que recebe, processa e armazena os dados em um banco de dados SQLite, e (3) **Frontend Web** que fornece visualização em tempo real e consulta histórica com filtros avançados.

O sistema fornece feedback visual (LEDs coloridos) e sonoro (buzzer) local, além de permitir que o usuário acesse os dados históricos através de um dashboard web responsivo, facilitando análise de padrões e geração de relatórios.

Slide 3: Arquitetura do Sistema

Título: Arquitetura em Três Camadas

A arquitetura do FireGuard IoT segue o modelo de três camadas: (1) **Camada de Sensores (Arduino)** — responsável pela leitura contínua do sensor, processamento de estados e feedback local; (2) **Camada de Dados (Backend Flask + SQLite)** — recebe dados via porta serial, valida, armazena e expõe uma API REST para consultas; (3) **Camada de Apresentação (Frontend Web)** — dashboard em tempo real via Web Serial API e página de histórico com filtros e estatísticas.

Essa separação permite escalabilidade, manutenibilidade e possibilita que diferentes componentes sejam desenvolvidos e testados independentemente.

Slide 4: Funcionalidades do Hardware

Título: Monitoramento Local: Sensor, LEDs e Buzzer

O Arduino UNO monitora continuamente o sensor de chama infravermelho, lendo valores de 0 a 1023. O sistema classifica cada leitura em três estados: **SEGURO** (valor > 700, LED verde), **ALERTA** (301–700, LED amarelo) e **PERIGO** (≤ 300 , LED vermelho + buzzer a 2500 Hz).

Quando o estado muda para PERIGO, o buzzer é ativado continuamente. O usuário pode silenciar o alarme pressionando um botão físico, ativando uma contagem regressiva de 10 segundos exibida em um display de 7 segmentos. Após a contagem, o sistema retoma o monitoramento automaticamente.

O sistema implementa histerese de 30 unidades nos limiares para evitar oscilação entre estados e debounce de 50 ms no botão para evitar acionamentos falsos.

Slide 5: Transmissão de Dados

Título: Comunicação Serial: Arduino → Backend

O Arduino transmite continuamente os dados do sensor e o estado atual via porta serial USB a 9600 baud. O formato das mensagens segue o padrão: `Sensor: [valor] | Estado: [SEGURO/ALERTA/PERIGO/SILENCIADO]`.

O Backend Python (servidor Flask) conecta-se à porta serial, lê essas mensagens em tempo real e as armazena no banco de dados SQLite com timestamp, valor do sensor e estado. Se a porta serial não estiver disponível, o sistema ativa um modo de simulação que gera dados fictícios para fins de demonstração e testes.

Slide 6: Banco de Dados

Título: Armazenamento Persistente em SQLite

O banco de dados SQLite armazena todas as leituras em uma tabela com os campos: ID (chave primária), Timestamp (data e hora), Sensor (valor 0–1023) e Estado (SEGURO/ALERTA/PERIGO/SILENCIADO).

Cada leitura é inserida no banco com timestamp automático, permitindo rastreabilidade completa. O banco é embutido (arquivo local) e não requer servidor externo, facilitando a portabilidade e implantação. O sistema foi testado com mais de 700 registros de exemplo, demonstrando performance adequada para consultas com filtros.

Slide 7: API REST do Backend

Título: Endpoints para Consulta de Dados

O Backend expõe uma API REST com os seguintes endpoints: (1) `GET /api/leituras` — retorna leituras com filtros opcionais (data, intervalo de datas, estado, limite); (2) `GET /api/leituras/resumo` — retorna estatísticas agregadas (total, min/max/média do sensor, contagem por estado); (3) `GET /api/leituras/datas` — retorna lista de datas disponíveis no banco; (4) `POST /api/leitura` — permite enviar uma leitura via JSON para integração com outros sistemas.

Todos os endpoints suportam CORS, permitindo acesso do frontend. Os filtros incluem atalhos de período (hoje, ontem, semana, mês) e filtros avançados por intervalo de datas e estado.

Slide 8: Dashboard em Tempo Real

Título: Visualização em Tempo Real via Web Serial API

O dashboard em tempo real (página `index.html` ou `serial.html`) conecta-se diretamente ao Arduino via Web Serial API (disponível em navegadores Chrome 89+ e Edge 89+). O usuário seleciona a porta serial USB e a velocidade (9600 baud), e o dashboard passa a exibir em tempo real: valor do sensor com gráfico de variação, estado atual com LEDs virtuais coloridos, contagem regressiva durante silenciamento e log de mensagens do Arduino.

O gráfico utiliza canvas para renderizar uma linha contínua do histórico de leituras com cores que mudam conforme o estado (verde para seguro, amarelo para alerta, vermelho para perigo). O dashboard é totalmente responsivo e funciona em dispositivos móveis.

Slide 9: Página de Histórico com Filtros

Título: Consulta e Análise de Dados Históricos

A página de histórico (`historico.html`) permite consultar os dados armazenados no banco de dados com múltiplos filtros: (1) **Atalhos de período** — Hoje, Ontem, Últimos 7 dias, Este mês, Todos; (2) **Filtros avançados** — Data início, Data fim, Estado (Seguro/Alerta/Perigo/Silenciado), Limite de registros (50 a 1000).

A página exibe: (1) **Tabela de registros** — com ID, timestamp, valor do sensor (com barra visual), e estado; (2) **Estatísticas resumidas** — total de leituras, quantidade por estado, valores mínimo, máximo e médio do sensor; (3) **Gráfico histórico** — linha contínua mostrando a variação do sensor ao longo do período consultado.

Slide 10: Exportação de Dados

Título: Geração de Relatórios em CSV

O usuário pode exportar os dados consultados no histórico para um arquivo no formato CSV (Comma-Separated Values) através de um botão na interface. O arquivo contém as colunas: ID, Timestamp, Sensor, Estado.

Esse recurso facilita a integração com ferramentas de análise externas (Excel, Python, Tableau) e permite que o usuário mantenha backups dos dados consultados. O arquivo é gerado dinamicamente no navegador sem necessidade de processamento no servidor.

Slide 11: Requisitos Funcionais

Título: O que o Sistema Faz (13 Requisitos Funcionais)

O sistema atende a 13 requisitos funcionais principais: (1) Leitura contínua do sensor; (2) Classificação automática em três estados; (3) Feedback visual local com LEDs; (4) Feedback sonoro com buzzer; (5) Silenciamento temporário via botão; (6) Contagem regressiva em display 7 segmentos; (7) Transmissão de dados via serial; (8) Armazenamento em banco de dados; (9) Dashboard em tempo real; (10) Consulta de histórico; (11) Filtros avançados de consulta; (12) Estatísticas resumidas; (13) Exportação para CSV.

Todos os requisitos foram implementados e testados com sucesso.

Slide 12: Requisitos Não Funcionais

Título: Como o Sistema Deve Operar (Qualidade e Desempenho)

O sistema atende a requisitos não funcionais em múltiplas categorias: **Eficiência** — leitura do sensor em ≤ 150 ms, resposta local em ≤ 200 ms; **Confiabilidade** — histerese de 30 unidades, debounce de 50 ms; **Segurança** — alarme a 2500 Hz, recuperação automática após silenciamento; **Usabilidade** — interface responsiva, feedback visual claro; **Portabilidade** — compatível com Arduino UNO, Flask, SQLite, navegadores modernos; **Manutenibilidade** — código modularizado com funções de responsabilidade única.

Slide 13: Diagrama de Casos de Uso

Título: Interações entre Atores e Funcionalidades

O diagrama de casos de uso mostra as interações entre três atores: (1) **Usuário** — interage com botão físico, dashboard em tempo real e página de histórico; (2) **Arduino** — monitora o

sensor, fornece feedback local e transmite dados; (3) **Backend** — recebe dados, armazena e expõe API.

Os 10 casos de uso principais incluem: monitorar ambiente, visualizar estado local, silenciar alarme, visualizar dashboard em tempo real, conectar ao Arduino via Web Serial, consultar histórico, filtrar dados, exportar CSV, armazenar leitura no banco e visualizar estatísticas.

Slide 14: Tecnologias Utilizadas

Título: Stack Tecnológico Completo

Hardware: Arduino UNO (ATmega328P), Sensor de chama infravermelho, LEDs RGB, Buzzer, Botão, Display 7 segmentos.

Firmware: C++ (Arduino IDE / PlatformIO), Máquina de estados, Tratamento de histerese e debounce.

Backend: Python 3, Flask (framework web), Flask-CORS (suporte a CORS), PySerial (leitura de porta serial), SQLite (banco de dados embutido).

Frontend: HTML5, CSS3, JavaScript ES6 vanilla, Web Serial API, Canvas (gráficos), Responsive Design.

Ferramentas: Git/GitHub (controle de versão), PlatformIO (gerenciamento de dependências), Markdown (documentação).

Slide 15: Resultados e Demonstração

Título: Sistema Funcional e Testado

O sistema foi implementado completamente e testado com sucesso. O banco de dados foi populado com mais de 700 registros de exemplo cobrindo os últimos 7 dias, permitindo demonstração realista de filtros e estatísticas.

O backend está rodando em `localhost:5000` e expõe a API REST. O frontend está acessível em `http://localhost:5000/historico.html` e permite consultar o histórico com diversos filtros. O dashboard em tempo real funciona via Web Serial API em navegadores Chrome/Edge modernos.

Todos os requisitos funcionais e não funcionais foram atendidos conforme especificado.

Slide 16: Documentação Entregue

Título: Artefatos do Projeto

O projeto inclui a seguinte documentação: (1) **Código-fonte completo** — Arduino (C++), Backend (Python), Frontend (HTML/CSS/JS); (2) **Requisitos** — Lista de 13 RF e 13 RNF com descrições detalhadas; (3) **Diagrama de casos de uso** — Ilustrando interações entre atores e funcionalidades; (4) **Documento PDF** — Consolidando requisitos, diagramas e código-fonte; (5) **Slides de apresentação** — Este material; (6) **Banco de dados de exemplo** — Com 700+ registros para demonstração.

Todos os arquivos estão disponíveis no repositório GitHub [richaferreira/Projeto_IoT](https://github.com/richaferreira/Projeto_IoT).

Slide 17: Conclusão

Título: FireGuard IoT: Solução Completa e Integrada

O FireGuard IoT demonstra a integração bem-sucedida de hardware, backend e frontend em um sistema de alarme de incêndio inteligente. O projeto atende a todos os critérios do Trabalho 2, incluindo monitoramento em tempo real, armazenamento em banco de dados, consulta histórica com filtros avançados, requisitos documentados e diagrama de casos de uso.

A arquitetura em três camadas permite escalabilidade e manutenibilidade. O código é modularizado, bem documentado e segue boas práticas de engenharia de software. O sistema está pronto para implantação e pode ser facilmente estendido com novas funcionalidades, como alertas por email, integração com IoT cloud ou suporte a múltiplos sensores.

Slide 18: Perguntas e Discussão

Título: Obrigado pela Atenção!

Perguntas, sugestões e discussões são bem-vindas.

Contato: Projeto disponível em https://github.com/richaferreira/Projeto_IoT

Demonstração ao vivo: Dashboard em tempo real e página de histórico com filtros.